ELECTRÓNICA DIGITAL I · SEGUNDA ENTREGA · CLASE 5 DE 5

# Integración y Quartus

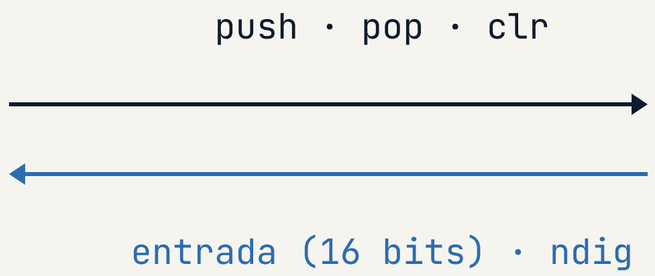
fsm_sesion, el top jerárquico, el testbench del sistema completo y lo que hay que revisar antes de programar la tarjeta.

CONTROL Y DATOS

# La FSM decide; los registros guardan

**fsm_sesion**

Unidad de control: en qué menú estamos, qué hacer con cada tecla.




**reg_desplazamiento**

Camino de datos: guarda los dígitos y cuántos hay.

El mismo patrón con los temporizadores: la FSM pide (ena, cargar, reiniciar) y lee el resultado (fin, cero). Ningún contador vive dentro de la FSM.

INTERFAZ ENTRE LAS DOS FSM · R4

# Las dos máquinas solo hablan por estas señales

| Señal | Dirección | Ancho | Significado |
|---|---|---|---|
| `cmd_nivel` | sesión → proceso | 1 | pulso: iniciar el llenado hasta el objetivo |
| `cmd_cancelar` | sesión → proceso | 1 | pulso: tecla C |
| `objetivo` | sesión → proceso | 3 | 25, 50, 75 o 100 % codificado |
| `man_bomba, man_comp` | sesión → proceso | 1 c/u | pedidos manuales (nivel, no pulso) |
| `ack_alarma` | sesión → proceso | 1 | pulso: el administrador reconoce la falla |
| `estado_proc` | proceso → sesión | 3 | estado actual, para el display y para apagar manuales |

FSM_SESION.VHD · PLANTILLA

# Seis estados hechos, cuatro para ustedes

| Estado | Qué hace | En la plantilla |
|---|---|---|
| BLOQ | captura 4 dígitos; D pide la clave de administrador | hecho |
| VERIF | compara 16 bits, cuenta intentos | hecho |
| CASTIGO | 30 s con el teclado ignorado (RF2) | hecho |
| MSG | muestra Err 2 s y vuelve a retorno | hecho |
| CLAVE1, CLAVE2 | nueva clave dos veces; guarda si coinciden (RF6) | hecho |
| **MENU** | A, C y * | **TODO 1** |
| **CAPT** | captura y valida el porcentaje objetivo (máx. 3 dígitos) | **TODO 2** |
| **MENU_ADM** | opciones 1 a 5 y * | **TODO 3** |
| **VER** | muestra el contador de vaciados | **TODO 4** |

CONTRASEÑAS · R9 Y RF6

# Un comparador de 16 bits, una sola vez

```
clave_ok <= '1' when
  (login_adm = '1'
    and entrada = CLAVE_ADM) or
  (login_adm = '0'
    and entrada = clave_usr)
  else '0';
```

**CLAVE_ADM**

Un generic: fija, se define al compilar.

**clave_usr**

Un registro de 16 bits: el administrador la cambia en CLAVE1 y CLAVE2.

**Al presionar #**

Solo en el estado VERIF se mira clave_ok.

TOP_TANQUE.VHD · R13

# El top solo conecta bloques

```
u_proceso : entity work.fsm_proceso
  port map (
    clk       => clk,
    rst       => rst,
    cmd_nivel => cmd_n,
    s5        => sens_s(4),
    nivel     => nivel,
    ...
    bomba_en  => bomba_s);
```

**entity work.x** instancia sin declarar component: menos código repetido.

Sin process. Solo instancias y asignaciones directas a pines.

Cada señal interna lleva el nombre de la Figura 1 de la guía.

TB_TOP_TANQUE.VHD · SECCIÓN 10 DE LA GUÍA

# Un testbench que se califica solo

**1**

### Modelos

Teclado con rebote y tanque que sube con la bomba y baja con la compuerta.

**2**

### Procedures

teclear("1234#"), tanque_en(2), verificar(cond, nombre).

**3**

### Casos

TC1 a TC12, uno por requerimiento, en un solo process.

**4**

### Resumen

Al final imprime cuántas verificaciones pasaron y cuántas fallaron.

```
teclear("1234#A75#");
wait until nivel_tq = 3 for 10 sec;
verificar(bomba_en = '0', "TC4 bomba apagada al llegar a 75 %");
```

TRUCO DEL TESTBENCH

# Leer los displays como texto

```
function seg2chr (s : std_logic_vector(6 downto 0))
  return character is
begin
  case s is
    when "1111110" => return '0';
    when "1001111" => return 'E';
    when "0000101" => return 'r';
    ...

verificar(pantalla = "Err ", "TC5 60 % rechazado");
```

El testbench traduce los 4 displays a un string de 4 letras y lo compara con lo que dice la tabla 3.4 de la guía.

Ojo: en 7 segmentos la S y el 5 tienen el mismo patrón. F-S se lee como "F-5 ".

QUARTUS · LECCIONES DE LA ENTREGA 1

# Cuatro líneas que la entrega 1 no tenía

```
# 1. el reloj en su pin real (50 MHz)
set_location_assignment PIN_R8 -to clk
# 2. toda la E/S a 3,3 V
set_global_assignment -name STRATIX_DEVICE_IO_STANDARD "3.3-V LVTTL"
# 3. pull-up en las columnas del teclado
set_instance_assignment -name WEAK_PULL_UP_RESISTOR ON -to columnas[0]
# 4. restricción de reloj (top_tanque.sdc)
create_clock -name clk -period 20.000 [get_ports {clk}]
```

El .qsf de la carpeta quartus/ trae todos los pines de la DE0-Nano. Verifíquenlos contra el manual y su cableado antes de programar.

ANTES DE PROGRAMAR · R12

# Cuatro números que van en el informe

| Qué | Dónde en Quartus | Meta |
|---|---|---|
| **Latches** | Analysis & Synthesis › Messages, buscar "latch" | cero |
| **Fmax** | Timing Analyzer › Slow 1200mV 85C › Fmax Summary | mayor que 50 MHz |
| **Elementos lógicos** | Flow Summary › Total logic elements | reportar |
| **Registros** | Flow Summary › Total registers | reportar y explicar |

La entrega 1 usó **19** elementos lógicos. ¿Cuántos usa la entrega 2 y cuáles bloques pesan más?

RTL Viewer: ¿se parece al diagrama de bloques? Si no, la jerarquía tiene un problema.

RÚBRICA · SECCIÓN 11 DE LA GUÍA

# Así se califica la entrega

| Criterio | Peso |
|---|---|
| Funcionamiento en la tarjeta (RF1 a RF10) | 30 % |
| Diseño de las FSM: diagramas, tablas, código | 20 % |
| Testbench y análisis de simulación | 20 % |
| Cumplimiento de R1 a R13 | 15 % |
| Protección de hardware y cálculos | 10 % |
| Calidad del informe | 5 % |

## Lo que más se olvida

Diagrama de fsm_sesion completo, no solo el código.

Captura de los 0 latches y del Fmax.

Esquemático de protección con sus cálculos.

Cualquier integrante explica cualquier bloque.

TALLER EN CLASE · ORDEN RECOMENDADO

# De la FSM a la tarjeta

**1 · fsm_sesion**

TODO 1 a 4 con su diagrama de tarea.

**2 · formateador y top**

Vistas que faltan y las instancias del top (TODO 1 a 8).

**3 · Testbench**

Primero los 4 casos hechos; luego TC3 y TC6 a TC12.

**4 · Quartus y tarjeta**

0 latches, Fmax, pines verificados. Recién ahí, programar.

Regla de oro: nada se programa en la tarjeta si el testbench no está en verde.